

Module 2: Search Algorithms & Game Playing

Uninformed Search (BFS/DFS/UCS/IDS), Heuristic A*, Minimax & Alpha-Beta Pruning

Module II: Problem Solving by Search Agents & Game Playing

1. Problem Formulation 5-Tuple (Initial State, Actions, Result, Goal, Cost)
2. State Space Graphs vs. Search Trees (Frontier & Explored Sets)
3. Uninformed Search Strategies (BFS, DFS, Uniform-Cost Search)
4. Depth-Limited Search (DLS) & Iterative Deepening Search (IDS) Proofs
5. Heuristic Search: Greedy Best-First Search & A* Search Algorithm
6. Admissibility & Consistency (Monotonicity) Formal Mathematical Proofs
7. Heuristic Dominance, Effective Branching Factor & IDA* / SMA*
8. Local Search: Hill Climbing, Plateau Escapes & Simulated Annealing ($e^{\Delta E/T}$)
9. Genetic Algorithms (Selection, Crossover, Mutation Operators)
10. Adversarial Search & Game Playing: Minimax Algorithm & Ply Traversal
11. Alpha-Beta Pruning Formulation & Step-by-Step Tree Tracing
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 1 & 2: Formulating State Space Search Problems

A search problem is formally specified as a **5-tuple** $\langle S_0, \text{Actions}(s), \text{Result}(s, a), \text{GoalTest}(s), c(s, a, s') \rangle$:

Initial State (S_0): The world state in which the agent starts (e.g., $\text{In}(\text{Arad})$).

Actions Set ($\text{Actions}(s)$): The legal moves executable from state s .

Transition Model ($\text{Result}(s, a)$): Specifies the successor state s' reached by executing action a in state s .

Goal Test ($\text{GoalTest}(s)$): A boolean predicate that determines whether state s is a goal state (e.g., $s = \text{In}(\text{Bucharest})$).

Path Cost Function ($c(s, a, s')$): Step cost of taking action a from s to reach s' . The path cost $g(n)$ is the sum of step costs from S_0 to node n .

Topic 3 & 4: Uninformed (Blind) Search Algorithms

Algorithm	Frontier Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
1. Breadth-First (BFS)	FIFO Queue	$O(b^d)$	$O(b^d)$	Yes (if $b < \infty$)	Yes (if step costs are equal)
2. Uniform-Cost (UCS)	Priority Queue by $g(n)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	Yes (if step cost $\geq \epsilon > 0$)	Yes (for general positive costs)
3. Depth-First (DFS)	LIFO Stack	$O(b^m)$	$O(b \cdot m)$	No (fails in infinite paths)	No
4. Depth-Limited (DLS)	LIFO Stack with limit l	$O(b^l)$	$O(b \cdot l)$	No (if $l < d$)	No
5. Iterative Deepening (IDS)	Iterative DLS ($l = 0, 1, 2, \dots$)	$O(b^d)$	$O(b \cdot d)$	Yes (if $b < \infty$)	Yes (if unit step costs)

Where b = branching factor, d = depth of shallowest goal, m = maximum depth of search tree, C^* = cost of optimal solution, ϵ = minimum positive step cost.

Why Iterative Deepening Search (IDS) is the Preferred Uninformed Search

IDS combines the **minimal linear memory requirement of DFS** ($O(bd)$) with the **completeness and optimality of BFS**.

Although nodes at depth k are re-generated multiple times, the bottom level dominates the sum:

$$N(\text{IDS}) = (d)b + (d-1)b^2 + \dots + (1)b^d = O(b^d)$$

For $b = 10$ and $d = 5$, BFS generates 111,111 nodes while IDS generates 123,456 nodes — an overhead of only 11% for massive memory savings!

Topic 5 & 6: Informed Heuristic Search & The A^* Algorithm

A^* search guides exploration using an evaluation function $f(n)$ that estimates the total cost of the cheapest path through node n to the goal:

$$f(n) = g(n) + h(n)$$

$g(n)$: Exact path cost from initial state S_0 to node n .

$h(n)$: Estimated cost of the cheapest path from node n to a goal state (Heuristic).

Mathematical Conditions for A^* Optimality

1. Admissibility (Required for Tree Search):

A heuristic $h(n)$ is **admissible** if it never overestimates the true cost to reach the goal:

$$\forall n, \quad 0 \leq h(n) \leq h^*(n)$$

Where $h^*(n)$ is the true optimal cost from n to goal.

2. Consistency / Monotonicity (Required for Graph Search):

A heuristic $h(n)$ is **consistent** if for every node n and every successor n' generated by action a :

$$h(n) \leq c(n, a, n') + h(n')$$

Triangle Inequality Property. Every consistent heuristic is strictly admissible (*Consistency* $\implies$ *Admissibility*).

Mathematical Proof: A^* is Optimal with an Admissible Heuristic (Tree Search)

Theorem: A^* tree search returns an optimal solution if $h(n)$ is admissible.

Proof by Contradiction:

1. Let G_2 be a suboptimal goal state on the frontier, with $g(G_2) > C^*$, where C^* is the cost of optimal goal G_1 .
2. Since G_2 is a goal state, $h(G_2) = 0$, so $f(G_2) = g(G_2) + 0 > C^*$.
3. Let n be an unexpanded node on the optimal path to G_1 . Since $h(n)$ is admissible:

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$$

4. Combining equations:

$$f(n) \leq C^* < f(G_2)$$

5. Since A^* always selects the frontier node with minimum f -value, node n will always be selected before G_2 . Thus, no suboptimal goal G_2 can ever be expanded before an optimal solution is found. ■

Topic 8: Local Search & Optimization (Simulated Annealing)

Simulated Annealing escapes local optima by combining hill climbing with stochastic exploration inspired by the metallurgical cooling of metals:

If a neighboring state s' improves the objective ($\Delta E = E(s') - E(s) > 0$), the move is always accepted.

If $\Delta E \leq 0$ (worse move), the move is accepted with Boltzmann probability:

$$P(\text{Accept}) = e^{\frac{\Delta E}{T}}$$

Where T is the temperature parameter dictated by an annealing schedule $T = \text{Schedule}(t)$. At high T , the agent acts like a random walk; as $T \rightarrow 0$, it behaves purely as greedy hill climbing.

Topic 10 & 11: Adversarial Search (Minimax & Alpha-Beta Pruning)

Minimax Value Formulation:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if TerminalTest}(s) \\ \max_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$

Alpha-Beta ($\alpha - \beta$) Pruning Rules

- **α (Alpha):** The best value (highest utility) found so far along the path to the current node for **MAX** (initially $-\infty$).
- **β (Beta):** The best value (lowest utility) found so far along the path to the current node for **MIN** (initially $+\infty$).
- **Pruning Condition:** Prune the remaining children of the current node whenever:

$$\alpha \geq \beta$$

- **Time Complexity with Perfect Move Ordering:** Reduces search complexity from $O(b^m)$ to $O(b^{m/2})$, effectively doubling the solvable search depth!



Top BIT Mesra Exam Questions & Answers (Module II)

Q1. State the 8-Puzzle problem formulation and compare the Manhattan Distance heuristic h_1 with the Misplaced Tiles heuristic h_2 . (8 Marks)

- **State:** 3×3 grid containing tiles 1 through 8 and one blank space.
- **Initial State:** Any arbitrary permutation of the 9 positions.
- **Actions:** Move blank space Left, Right, Up, Down.
- **Goal Test:** Tiles arranged in ascending numerical order.
- **Path Cost:** Each move costs 1 unit ($g(n) = \text{depth}$).

Comparison of Heuristics:

1. *Misplaced Tiles* (h_1): Counts number of tiles not in their goal position.
2. *Manhattan Distance* (h_2): Sum of vertical and horizontal grid distances of each tile from its goal position:

$$h_2 = \sum_{i=1}^8 (|x_i - x_i^*| + |y_i - y_i^*|)$$

3. **Dominance:** $\forall n, h_2(n) \geq h_1(n)$. Both are admissible, but h_2 strictly **dominates** h_1 , expanding fewer search nodes while guaranteeing optimal solutions.

Q2. Trace Alpha-Beta Pruning on a 3-level game tree and explain why pruned nodes do not affect the root decision. (10 Marks)

Alpha-Beta pruning computes the exact same minimax decision as standard Minimax without evaluating branches that cannot alter the final choice.

When a MIN node evaluates a child with value $v \leq \alpha$, MIN will pick a value $\leq v \leq \alpha$. But MAX (higher in the tree) already has a confirmed choice with value α . Hence MAX will never choose the branch leading to this MIN node, making further exploration of its remaining children mathematically pointless.